

UptimeHub

Complete End-to-End Project Explanation

Source material for NotebookLM video/audio walkthrough

This document explains UptimeHub by following a realistic user journey: a user creates a website monitor, the request travels through Azure and Kubernetes, background services schedule and execute checks, results are stored in PostgreSQL, and Prometheus/Grafana/Alertmanager provide observability.

1. What is UptimeHub?

UptimeHub is a cloud-native website monitoring platform. A user gives UptimeHub a website or HTTP endpoint and tells it how often the endpoint should be checked.

Example

Website: `https://www.google.com`

Check every: 60 seconds

Monitoring: Active

UptimeHub continuously checks the website and records whether it is UP or DOWN, the HTTP status code, response time, and the last checked time.

The project is more than a Python script. It combines Microsoft Azure, Azure Kubernetes Service (AKS), Azure Application Gateway, Docker, FastAPI, PostgreSQL, Redis, Terraform, GitHub, Azure DevOps, Azure Container Registry, Azure Key Vault, Azure Workload Identity, Prometheus, Grafana, Alertmanager, and Trivy.

2. The Complete UptimeHub Architecture

UptimeHub is easiest to understand as several connected flows.

USER / APPLICATION FLOW

User



Internet



Azure Application Gateway



AGIC / Kubernetes Ingress



Kubernetes Service



FastAPI Backend Pods



PostgreSQL

BACKGROUND MONITORING FLOW

PostgreSQL



Scheduler

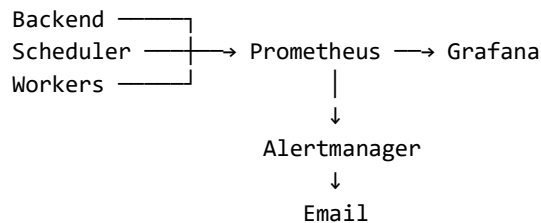


```

Redis Queue
↓
Workers
↓
Monitored Website
↓
Worker
↓
PostgreSQL

```

OBSERVABILITY FLOW



APPLICATION DELIVERY

GitHub → Azure DevOps → Tests/Scans → Docker → ACR → AKS

INFRASTRUCTURE DELIVERY

Terraform → Validate → Security Scan → Plan → Approval → Apply → Azure

3. Step 1 — A User Creates a Monitor

Imagine a user wants to monitor Google. In the current development environment, the FastAPI Swagger interface exposes the UptimeHub API.

```

POST    /api/v1/monitors
GET     /api/v1/monitors
GET     /api/v1/monitors/{id}
PUT     /api/v1/monitors/{id}
DELETE  /api/v1/monitors/{id}
POST    /api/v1/monitors/{id}/check
GET     /health

```

The user selects POST /api/v1/monitors and submits a monitor.

```
{ "name": "Google", "url": "https://www.google.com", "interval_seconds": 60, "is_active": true }
```

This means: monitor Google, check it every 60 seconds, and keep monitoring enabled. When Execute is pressed, an HTTP request starts moving through UptimeHub.

4. Step 2 — Azure Application Gateway

The request travels over the Internet to the public IP of Azure Application Gateway. Application Gateway is the external HTTP entry point into UptimeHub.

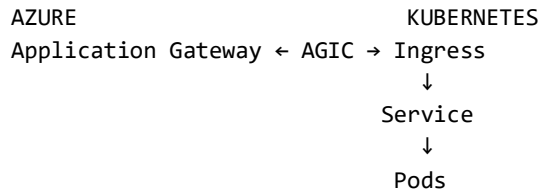
User Browser → Internet → Public IP → Azure Application Gateway

The user does not connect directly to an AKS Pod. Application Gateway provides a controlled entry point before traffic reaches Kubernetes.

The current development environment uses HTTP through a public IP. Custom DNS, trusted HTTPS/TLS, and WAF are production enhancements rather than features to present as already implemented.

5. Step 3 — AGIC and Kubernetes Ingress

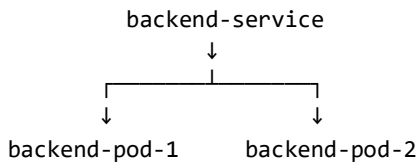
Azure Application Gateway and Kubernetes Ingress are different components. Application Gateway is an Azure networking resource. Ingress is a Kubernetes routing object. AGIC, the Application Gateway Ingress Controller, connects the two.



Ingress describes where HTTP traffic should go, for example / to backend-service. AGIC watches that configuration and configures Application Gateway so traffic can reach the Kubernetes backend.

6. Step 4 — Kubernetes Service

Inside AKS, multiple FastAPI backend Pods can exist. Pods are temporary and can be recreated with different IP addresses, so external components should not depend on individual Pod IPs.



A Deployment manages Pods. A Service gives those Pods a stable network endpoint. Ingress defines HTTP routing to the Service. The request path is Application Gateway → Ingress → Service → FastAPI Pod.

7. Step 5 — FastAPI Processes the Request

The backend is Python/FastAPI running in a Docker container inside an AKS Pod. The container serves the application on port 8000.

AKS → Backend Pod → Docker Container → Python/FastAPI → Port 8000

FastAPI receives POST /api/v1/monitors, validates the request, applies application logic, and uses SQLAlchemy to communicate with PostgreSQL.

8. Step 6 — PostgreSQL Stores the Monitor

PostgreSQL is UptimeHub's persistent database. A newly created monitor can initially have an unknown status because creating a monitor and checking a monitor are separate operations.

```
ID: 3
Name: Google
URL: https://www.google.com
Interval: 60 seconds
```

```
Active: true
Current Status: unknown
Last HTTP Status: null
Last Response Time: null
Last Checked: null
```

If a FastAPI Pod is restarted or replaced, the monitor remains in PostgreSQL. Pods run application processes; PostgreSQL stores persistent application data.

9. Step 7 — Background Monitoring

UptimeHub does not require a user to press Check every 60 seconds. Background monitoring is separated into Scheduler, Redis, and Worker responsibilities.

```
Scheduler = decides WHEN a monitor needs checking
Redis      = holds monitoring jobs and deduplication locks
Worker     = performs the actual website check
```

10. Step 8 — The Scheduler

The Scheduler queries PostgreSQL to determine which active monitors are due. If monitor ID 3 is due, the Scheduler does not contact Google directly. It creates a job for a Worker.

```
Scheduler → PostgreSQL
Question: "Which active monitors are due?"
Result: Monitor ID 3 – Google – due now
```

11. Step 9 — Redis Job Queue and Deduplication

The Scheduler places the monitor ID into Redis. Redis decouples scheduling from execution, allowing Workers to process jobs independently.

```
Scheduler → Redis Queue
```

```
[ monitor_id = 3 ]
[ monitor_id = 8 ]
[ monitor_id = 4 ]
```

Redis also supports deduplication. A lock prevents the same monitor from being unnecessarily queued multiple times during the relevant scheduling window.

```
Redis responsibilities:
1. Job Queue
2. Job Deduplication
```

12. Step 10 — A Worker Takes the Job

Worker Pods wait for jobs from Redis. A Worker receives `monitor_id = 3`, loads monitor 3 from PostgreSQL, and learns that the target URL is `https://www.google.com`.

```
Redis → Worker → PostgreSQL
          ↓
        Monitor ID 3
```

Google
https://www.google.com

13. Step 11 — The Worker Checks the Website

The Worker performs the real HTTP request across the Internet. Suppose Google responds with HTTP 200 in 457 milliseconds.

```
Worker → Internet → https://www.google.com
                        ↓
                    HTTP 200
                    457 ms
```

The Worker interprets the successful result as an UP status and records the HTTP code and response time.

14. Step 12 — PostgreSQL is Updated

The Worker updates the persistent monitor record.

```
BEFORE
Status: unknown
HTTP status: null
Response time: null

AFTER
Status: UP
HTTP status: 200
Response time: 457 ms
Last checked: current timestamp
```

This completes the central UptimeHub journey: User → Application Gateway → Ingress → Service → FastAPI → PostgreSQL → Scheduler → Redis → Worker → monitored website → PostgreSQL.

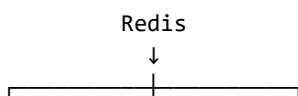
15. Step 13 — When a Website is Down

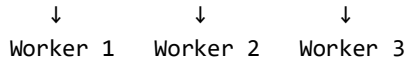
The scheduling path remains the same when a site is unavailable. The Worker may receive a timeout, connection error, or unsuccessful result and records the monitor as DOWN.

```
Scheduler → Redis → Worker → Website
                        ↓
                    Timeout / Error
                        ↓
                    Status = DOWN
```

16. Why Multiple Workers?

The Redis queue allows monitoring execution to scale horizontally. With more work, multiple Workers can consume different jobs.





Kubernetes HPA configuration can adjust replicas according to configured resource metrics and limits. This architecture separates job production from job execution.

17. Kubernetes Reliability and Workload Management

The Kubernetes configuration uses Deployments, Services, Ingress, ConfigMaps, ServiceAccounts, NetworkPolicies, health probes, CPU/memory requests and limits, Horizontal Pod Autoscalers, and Pod Disruption Budgets.

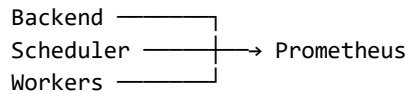
The backend exposes /health for health checking. Kubernetes uses orchestration features to manage containerized UptimeHub workloads, networking, availability, and scaling behavior.

18. Prometheus — Monitoring UptimeHub Itself

UptimeHub must monitor not only external websites but also its own operational behavior. The backend, Scheduler, and Workers expose Prometheus metrics.

```
uptimehub_http_requests_total
uptimehub_http_request_duration_seconds
uptimehub_monitor_checks_total
uptimehub_monitor_check_failures_total
uptimehub_monitor_response_time_seconds
uptimehub_monitor_queue_size
```

Prometheus periodically collects these metrics and stores time-series data. It answers questions such as API request volume, request latency, monitor failures, website response times, and queue size.



19. Grafana — Visualizing Metrics

Grafana queries Prometheus and presents metrics as human-friendly dashboards. UptimeHub dashboards include API activity, requests per second, P95 API latency, and monitor checks by status.

```
Prometheus = collect, store, and query metrics
Grafana    = visualize those metrics
```

20. Alertmanager — Notifications

Prometheus evaluates alert rules. When a condition is triggered, Alertmanager handles notification delivery, including email.

```
Prometheus → Alertmanager → Email
```

A simple mental model is: Prometheus detects that an alert condition is true; Alertmanager decides how the notification should be delivered.

21. Azure Key Vault and Workload Identity

UptimeHub requires sensitive PostgreSQL and Redis connection information. These application secrets are stored in Azure Key Vault rather than committed directly into application source code.

```
Key Vault secrets:
- database-url
- redis-url
```

The Kubernetes ServiceAccount uptimehub-app participates in Azure Workload Identity. This allows the workload to authenticate to Azure without embedding a traditional Azure client secret in the Pod. The Key Vault CSI integration makes the required secrets available to the workload.

```
UptimeHub Pod → ServiceAccount → Workload Identity → Azure → Key Vault
```

22. Azure Container Registry (ACR)

The FastAPI application is packaged as a Docker image. Azure Container Registry is UptimeHub's private container image registry.

```
Python Source → Docker Build → ACR → AKS → Pod
```

AKS has permission to pull the required application images from ACR.

23. Application CI/CD Pipeline

Application changes flow from GitHub through Azure DevOps. The pipeline automates testing, security scanning, image creation, registry publishing, Kubernetes deployment, and a smoke check.

```
Developer
↓
GitHub
↓
Azure DevOps
├─ Tests
├─ Trivy repository/IaC security scan
├─ Docker build
├─ Container image security scan
├─ Push image to ACR
├─ Deploy to AKS
└─ Smoke check
```

This is the application delivery path. It is separate from the Terraform infrastructure pipeline.

24. Terraform Infrastructure Pipeline

Terraform defines Azure infrastructure as code. The project uses modules for networking, AKS, ACR, PostgreSQL, Redis, Key Vault, and Application Gateway.

```
Terraform Code → Validate → Trivy Security Scan → Plan → Approval → Apply → Azure
```

Terraform remote state is stored in Azure Storage. The Plan stage shows intended infrastructure changes before Apply. Application code changes use the application pipeline; Azure infrastructure changes use the Terraform pipeline.

25. Azure Network Design

The development VNet uses address space 10.10.0.0/16 and separates major responsibilities into subnets.

```
VNet 10.10.0.0/16
├─ AKS subnet                10.10.0.0/20
├─ Application Gateway subnet 10.10.16.0/24
├─ Private Endpoints subnet  10.10.17.0/24
└─ PostgreSQL subnet         10.10.18.0/24
```

The network design separates application compute, ingress, private endpoint connectivity, and PostgreSQL networking. PostgreSQL, Redis, and Key Vault use private connectivity patterns in the Azure environment.

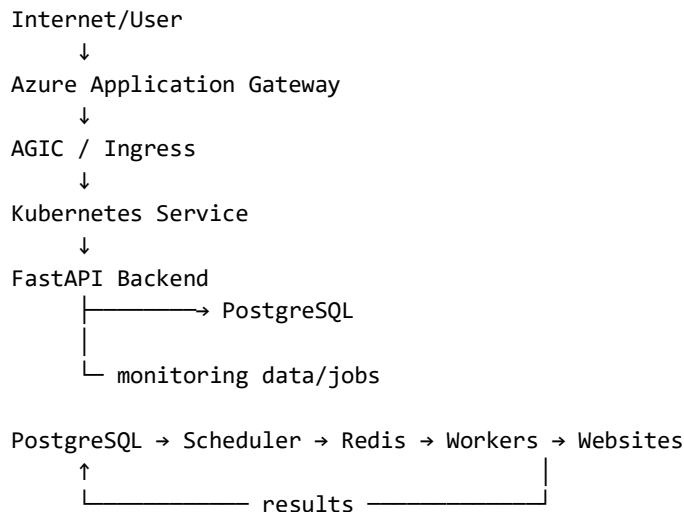
26. Complete End-to-End User Story

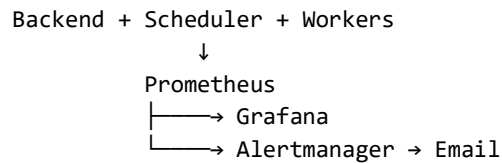
A complete UptimeHub user story can be summarized as follows.

1. User creates a Google monitor.
2. Request reaches Azure Application Gateway.
3. AGIC/Ingress routes it into AKS.
4. Kubernetes Service sends it to a FastAPI Pod.
5. FastAPI validates the request.
6. PostgreSQL stores the monitor.
7. Scheduler determines when it is due.
8. Scheduler queues monitor ID in Redis.
9. Worker consumes the job.
10. Worker loads the URL from PostgreSQL.
11. Worker checks Google over HTTP.
12. Google returns HTTP 200 and response time.
13. Worker updates PostgreSQL with UP/200/response time.
14. Prometheus collects UptimeHub operational metrics.
15. Grafana visualizes the metrics.
16. Alertmanager can deliver alert notifications when alert conditions are triggered.

27. The Two Diagrams to Remember

Runtime architecture:





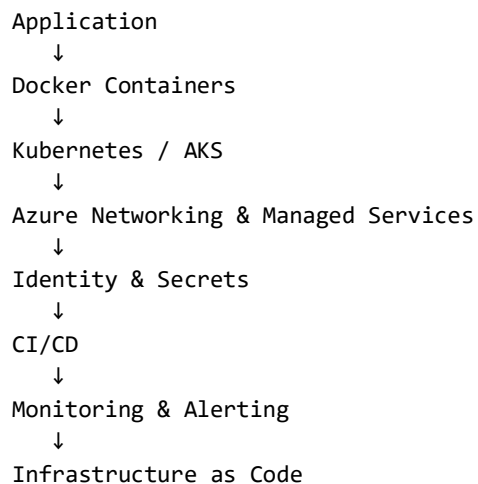
Engineering and delivery architecture:

GitHub → Azure DevOps → Tests/Trivy → Docker → ACR → AKS

Terraform → Validate/Trivy → Plan → Approval → Apply → Azure

28. Final Understanding

UptimeHub connects application development, containers, Kubernetes, cloud infrastructure, networking, identity, secrets management, CI/CD, Infrastructure as Code, metrics, dashboards, and alerting into one project.



The most important concept is not memorizing every YAML or Terraform resource. It is understanding how a user's request and a monitoring job move through the system, and which component owns each responsibility.